

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn import metrics
import seaborn as sns
```

```
data = {
'Brightness': [40, 50, 60, 10, 70, 60, 25],
'Saturation': [20, 50, 90, 25, 70, 10, 80],
'Class': ['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']}
df = pd.DataFrame(data)
df
```

	Brightness	Saturation	Class	
0	40	20	Red	
1	50	50	Blue	
2	60	90	Blue	
3	10	25	Red	
4	70	70	Blue	
5	60	10	Red	
6	25	80	Blue	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
x=df.iloc[:, :-1].values
y=df.iloc[:, -1].values
```

```
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.3, random_state=0, stratify=y)
```

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

```
results=[]
for i in range(1, len(x_train) + 1):
    model = KNeighborsClassifier(n_neighbors=i, metric='minkowski', p=2)
    model.fit(x_train, y_train)
    y_pred=model.predict(x_test)
    Accuracy_score=metrics.accuracy_score(y_test, y_pred)
    results.append (Accuracy_score)
print('KNN [ minkowski ]')
print('for n neighbor=' + str(i) + ' : ')
conf_mat=metrics.confusion_matrix(y_test, y_pred)
print('\n Confusion Matrix: ', conf_mat)
print('Accuracy Score : ', Accuracy_score)
print('Accuracy in Percentage : ',
int (Accuracy_score*100),'%')
print('\n',classification_report(y_test, y_pred))
print(results)
```

```
KNN [ minkowski ]
for n neighbor=4 :
```

```
Confusion Matrix: [[2 0]
[1 0]]
Accuracy Score : 0.6666666666666666
Accuracy in Percentage : 66 %
```

	precision	recall	f1-score	support
Blue	0.67	1.00	0.80	2
Red	0.00	0.00	0.00	1
accuracy			0.67	3

```
macro avg    0.33    0.50    0.40    3
weighted avg 0.44    0.67    0.53    3
```

```
[0.6666666666666666, 0.6666666666666666, 0.6666666666666666, 0.6666666666666666]
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in true labels but not predicted: 'Blue'
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in true labels but not predicted: 'Blue'
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in true labels but not predicted: 'Blue'
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
conf_mat=pd.crosstab(y_test, y_pred, rownames=['Actual'],colnames=['Predicted'])
sns.heatmap(conf_mat, annot=True).set(title='KNN [minkowski, neighbor=5 ]')
```

```
[Text(0.5, 1.0, 'KNN [minkowski, neighbor=5 ]')]
```

